

BITÁCORA DE EJECUCIÓN Y CIERRE — SPRINT 4

Foco del Incremento: Reservas, Historial y Comunicación — Flujo de booking dedicado, historial personal de reservas, botón flotante de WhatsApp y notificación por correo electrónico **Stack Tecnológico:** Java 21 / Spring Boot 3.5 / Spring Security 6 / JavaMailSender / MariaDB / React 19 / Vite / Testcontainers / SpringDoc OpenAPI

1. Resumen del Incremento (Scope)

El Sprint 4 completó la funcionalidad central de reservas de la plataforma. Se reemplazó el modal de reserva provisional (Sprint 3) por un flujo de página dedicada en dos columnas (`/booking/:lodgingId`), que presenta el resumen completo del alojamiento junto al formulario de reserva. Al confirmar, el usuario es redirigido a una pantalla de confirmación con los detalles de la estadía.

Se implementó el historial personal de reservas (`/my-reservations`) con ordenamiento por fecha de entrada descendente, protegido por el nuevo guard `RequireAuth` . Se incorporó un botón flotante de WhatsApp configurable por variable de entorno, y se desarrolló un servicio de email SMTP real (`SmtpEmailServiceImpl`) que envía confirmaciones HTML al huésped mediante Mailtrap, activable de forma independiente al `ConsoleEmailServiceImpl` ya existente.

En el backend, se corrigió el endpoint de disponibilidad para retornar los rangos ocupados (`occupiedRanges`) sin necesidad de recibir fechas, permitiendo el bloqueo visual del calendario al cargar la página. Se agregó validación de reserva confirmada antes de permitir puntuaciones (`RatingServiceImpl`), y se añadieron 5 nuevos tests de integración con Testcontainers que llevan el total a 144 tests en verde.

Como mejora complementaria al panel de administración, se extendió `LodgingFormModal` con soporte de edición (prop `lodging` opcional + `PUT`) y se incorporaron los botones "Editar" por fila en la tabla de alojamientos.

2. Arquitectura del Sistema e Integración

2.1. Backend (Spring Boot + Spring Security 6)

Se expandió la arquitectura existente sin introducir nuevos módulos estructurales. Los cambios se concentran en la capa de servicio (email), repositorios (nuevas queries), entidades (nuevos campos), y el endpoint de disponibilidad:

```
Controller → Service (Interface + Impl) → Repository → Entity / DTO
```

Matriz de Componentes Introducidos o Modificados:

Módulo	Entidad / DTO Afectado	Cambio en Capa de Servicio	Cambio en Controller
Reservas	Reservation (+guestPhone), ReservationResponse, CreateReservationRequest	ReservationService / ReservationServiceImpl: historial propio	ReservationController: GET /api/reservations/my, corrección de availability
Email	—	SmtEmailServiceImpl (nueva, @Primary), ConsoleEmailServiceImpl (renombrado)	—
Alojamientos	LodgingDTO (+averageRating, +ratingCount)	LodgingServiceImpl.enrichWithRatings()	—
Categorías	Category (+imageUrl)	—	—
Reseñas	—	RatingServiceImpl: validación de reserva CONFIRMED previa	—

- **SmtEmailServiceImpl con @Primary** : Se optó por @Primary en lugar de @ConditionalOnMissingBean porque esta última anotación solo evalúa confiablemente en clases @Configuration . Aplicada a clases escaneadas con @Service , el orden de evaluación del contexto Spring no está garantizado. @Primary resuelve la ambigüedad de forma explícita y determinista.
- **Endpoint de disponibilidad corregido**: GET /api/lodgings/{id}/availability acepta checkIn / checkOut opcionales. Sin parámetros, devuelve todos los rangos CONFIRMED (lista occupiedRanges) para bloqueo visual del calendario. Con parámetros, además calcula el boolean available .
- **Validación de reseñas**: RatingServiceImpl.createRating() verifica existsByUserIdAndLodgingIdAndStatus(CONFIRMED) antes de persistir. Lanza IllegalArgumentException si el usuario no tiene reserva confirmada.
- **Nuevas queries en repositorios**:
 - ReservationRepository.findByUserIdOrderByCheckInDesc(Long userId)
 - ReservationRepository.existsByUserIdAndLodgingIdAndStatus(...)
 - RatingRepository.countByLodgingId(Long lodgingId)

2.2. Frontend (React + Vite)

Evolución de la SPA con tres páginas nuevas, dos componentes nuevos y múltiples actualizaciones de componentes existentes:

```
src/
├── components/
│   ├── RequireAuth.jsx           (nuevo – route guard para usuarios autenticados)
│   └── WhatsAppButton/
│       └── WhatsAppButton.jsx    (nuevo – botón flotante fijo bottom-right)
├── pages/
│   ├── Booking/
│   │   ├── BookingPage.jsx      (nuevo – formulario de reserva en dos columnas)
│   │   ├── BookingPage.css      (nuevo)
│   │   └── BookingConfirmation.jsx (nuevo – confirmación post-reserva)
│   └── MyReservations/
│       └── MyReservationsPage.jsx (nuevo – historial de reservas del usuario)
```

- **BookingPage en dos columnas:** La columna izquierda muestra nombre, ciudad, precio, imagen, descripción y features del alojamiento. La columna derecha contiene el formulario con datos del usuario (nombre, apellido, email como read-only; teléfono editable), date pickers con fechas bloqueadas y cálculo dinámico del total.
- **Carga de `occupiedRanges` al montar:** `BookingPage` llama a `GET /api/lodgings/{id}/availability` al montar, sin necesitar que el usuario seleccione fechas primero. Esto permite deshabilitar visualmente los rangos ocupados en el calendario desde el inicio.
- **RequireAuth como route guard:** Redirige a `/login` con mensaje "Necesitás iniciar sesión para continuar. Si no tenés cuenta, podés registrarte." y preserva la ruta destino en el state (`from`) para redirigir al volver.
- **WhatsAppButton configurable:** Lee `VITE_WHATSAPP_NUMBER` del entorno. Si no está definida, el componente no renderiza. El enlace usa `wa.me/{número}` con mensaje pre-cargado. Posición fija: `bottom: 24px; right: 24px` . No requiere autenticación.
- **Eliminado:** `components/Reservation/ReservationModal.jsx` y su CSS — reemplazados por el flujo de página dedicada.
- **Header:** Agrega link "Mis reservas" para usuarios autenticados.
- **ProductDetail:** El botón "Reservar" navega a `/booking/:id` pasando fechas por state. Si el usuario no está autenticado, muestra un link a `/login` .
- **ShareModal:** Agrega Instagram (color `#E1306C`). Todos los íconos de redes sociales reemplazados por SVGs propios, eliminando dependencias externas.
- **LodgingFormModal extendido:** Acepta prop `lodging` opcional. Si está presente, pre-llena el formulario y usa `PUT /api/lodgings/{id}` . El título cambia a "Editar alojamiento". `hasChanges` compara los valores actuales contra los originales para mostrar el `ConfirmDialog` solo cuando hay cambios reales.
- **LodgingsTable y AdminLodgings:** Agrega botón "Editar" por fila y manejo del estado `editingLodging` .
- **SearchResults:** Agrega secciones "Categorías" y "Te puede interesar" bajo los resultados.

3. Trazabilidad de Historias de Usuario (User Stories)

ID	Historia de Usuario	Componente / Vista UI	Endpoint Backend	Criterio de Aceptación / Estado
US #30	Seleccionar fecha de check-in/check-out con fechas ocupadas bloqueadas.	BookingPage.jsx	GET /api/lodgings/{id}/availability	Calendario con rangos CONFIRMED deshabilitados cargados al montar la página.
US #31	Visualizar detalles del alojamiento al iniciar la reserva.	BookingPage.jsx	GET /api/lodgings/{id}	Nombre, ciudad, precio, imagen, descripción y features visibles en columna izquierda.
US #32	Realizar la reserva y recibir confirmación en pantalla.	BookingPage.jsx, BookingConfirmation.jsx	POST /api/reservations	HTTP 201 → redirige a /booking/confirmation CON nombre, fechas, huésped y total.
US #33	Acceder al historial personal de reservas.	MyReservationsPage.jsx	GET /api/reservations/my	Lista ordenada por checkIn DESC con alojamiento, fechas, estado y total. Protegida por RequireAuth.
US #34	Iniciar conversación de WhatsApp con mensaje pre-cargado.	WhatsAppButton.jsx	N/A (Frontend)	Botón flotante visible para todos. Enlace wa.me con mensaje. Oculto si VITE_WHATSAPP_NUMBER no definida.
US #35	Recibir email de confirmación al realizar una reserva.	BookingConfirmation.jsx	POST /api/reservations (dispara EmailService)	SmtEmailServiceImpl envía email HTML con datos de reserva. ConsoleEmailServiceImpl loguea en consola por defecto.

4. Catálogo de Endpoints Nuevos / Modificados

4.1. Reservas

Método	Endpoint	Acceso (RBAC)	Descripción
POST	/api/reservations	Autenticado	Crear reserva (ya existía — ahora incluye guestPhone)
GET	/api/reservations/my	Autenticado	Historial del usuario autenticado, ordenado por checkIn DESC
GET	/api/lodgings/{id}/availability	Público	Disponibilidad. Sin params: retorna occupiedRanges (todos los CONFIRMED). Con checkIn/checkOut: además calcula available.

4.2. Alojamientos (modificación)

Método	Endpoint	Acceso (RBAC)	Descripción
PUT	/api/lodgings/{id}	ADMIN	Ya existía. Usado ahora desde LodgingFormModal en modo edición.
GET	/api/lodgings/{id}	Público	Ya existía. LodgingDTO ahora incluye averageRating y ratingCount.

5. Modelo de Datos

Modificaciones en Entidades Existentes

RESERVATION id (PK, Long) lodging_id (FK) user_id (FK) check_in (NN) check_out (NN) guest_name (NN) guest_email (NN) + guest_phone (nullable) total_price (NN) status (ENUM) version (@Version)	CATEGORY id (PK, Long) name (NN) description icon + image_url (nullable)
	LODGING DTO (+) (campos existentes) + averageRating (Double) + ratingCount (Integer) (calculados en servicio)

- **Reservation:** Se agrega `guestPhone` (String, nullable) para el campo de teléfono del formulario de reserva, utilizado también como punto de contacto para WhatsApp.
- **Category:** Se agrega `imageUrl` (String, nullable) para soporte de imagen representativa de la categoría en `CategoryCard` y secciones de búsqueda.
- **LodgingDTO:** `averageRating` (Double) y `ratingCount` (Integer) se calculan en `LodgingServiceImpl.enrichWithRatings()` usando `RatingRepository.countByLodgingId()` y la media de puntuaciones. No son columnas persistidas en la tabla `lodging` — son campos computados en la capa de servicio y proyectados al DTO.

Nuevas Queries en Repositorios

Repositorio	Método	Propósito
ReservationRepository	<code>findByUserIdOrderByCheckInDesc(Long)</code>	Historial de reservas del usuario (GET <code>/api/reservations/my</code>)
ReservationRepository	<code>existsByUserIdAndLodgingIdAndStatus(Long, Long, Status)</code>	Valida reserva CONFIRMED antes de permitir puntuación
RatingRepository	<code>countByLodgingId(Long)</code>	Conteo de reseñas para calcular <code>ratingCount</code> en el DTO

6. Decisiones Técnicas Clave

- **Página dedicada vs modal para reservas:** Se reemplazó el modal de reserva de Sprint 3 por la página `/booking/:lodgingId`. El criterio de la US #31 requiere que la información completa del alojamiento sea visible durante el proceso de reserva, lo que es difícil de lograr en un modal sin saturarlo. La página dedicada también mejora la experiencia en dispositivos móviles donde los modales con scroll interno resultan incómodos.
- **occupiedRanges sin parámetros de fecha:** El endpoint `GET /api/lodgings/{id}/availability` ahora acepta llamadas sin `checkIn` / `checkOut`. En ese caso retorna la lista completa de rangos CONFIRMED. Esto

permite que `BookingPage` bloquee las fechas en el calendar picker al cargar, sin esperar que el usuario seleccione un rango primero — eliminando la ventana donde el usuario ve fechas ocupadas como disponibles.

- **@Primary sobre @ConditionalOnMissingBean para EmailService:** `@ConditionalOnMissingBean` aplicado directamente en clases `@Service` tiene comportamiento no determinista en Spring Boot porque el scanner de componentes no garantiza el orden de evaluación de beans. La solución correcta y predecible es `@Primary` en `SmtEmailServiceImpl` : cuando ambas implementaciones coexisten en el contexto, Spring inyecta la marcada como `@Primary` . El toggle se controla externamente con `app.mail.smtp.enabled` .
- **WhatsApp como enlace `wa.me` (sin Business API):** La API oficial de WhatsApp Business requiere cuenta verificada, aprobación de Meta y número dedicado. El enlace `wa.me/{número}` abre directamente la conversación en cualquier dispositivo (web o app móvil) sin dependencias de backend. El mensaje pre-cargado se define en el componente. La confirmación de entrega del mensaje es responsabilidad del sistema de WhatsApp.
- **Edición de alojamiento reutilizando `LodgingFormModal` :** En lugar de crear un componente `EditLodgingModal` separado, se extendió el modal existente con una prop `lodging` opcional. Si el prop está presente, el formulario se inicializa con sus valores, el submit usa `PUT` , y el título cambia. El campo `hasChanges` compara el estado actual contra los valores originales para no mostrar el `ConfirmDialog` si el usuario no modificó nada. Esta estrategia minimiza duplicación y mantiene el componente en un único lugar.

7. Testing

- **144 tests backend:** Todos en verde. Distribuidos en tests unitarios (JUnit 5 + Mockito) y de integración (MockMvc + Testcontainers con MariaDB 10.11). Se agregaron 5 tests nuevos en `ReservationControllerIntegrationTest` :
 1. Crear reserva válida → HTTP 201
 2. Crear reserva en fechas solapadas → HTTP 409 Conflict
 3. Crear reserva sin autenticación → HTTP 401/403
 4. Historial propio del usuario autenticado → HTTP 200
 5. Disponibilidad con `occupiedRanges` → HTTP 200
- **Cobertura del incremento:** Creación de reservas, solapamiento de fechas, seguridad de endpoints, historial por usuario, y retorno de rangos ocupados.
- **Frontend:** Sin test runner configurado. `npm run build` exitoso con 0 errores y 0 warnings. Las validaciones funcionales son manuales (ver Plan de Pruebas Sprint 4).

8. Limitaciones Conocidas y Deuda Técnica Controlada

1. **WhatsApp Business API:** El enlace `wa.me` no provee confirmación de envío ni manejo de errores desde la aplicación. La integración con la API oficial de WhatsApp Business (Meta) requiere cuenta verificada, número dedicado y proceso de aprobación — queda como mejora futura.
2. **Email SMTP desactivado por defecto:** `ConsoleEmailServiceImpl` es el default de desarrollo. Para activar el envío real se requiere `MAIL_SMTP_ENABLED=true` más las credenciales de Mailtrap (`MAILTRAP_HOST` , `MAILTRAP_PORT` , `MAILTRAP_USERNAME` , `MAILTRAP_PASSWORD`) en las variables de entorno.
3. **Frontend sin tests automatizados:** No hay test runner configurado en el frontend. La cobertura es manual + build sin errores.
4. **Precios por temporada:** El total de reserva se calcula como `días × pricePerNight` . No hay soporte para tarifas variables por temporada o fin de semana.
5. **Refresh tokens:** El JWT expira a las 8 horas sin mecanismo de renovación, forzando reautenticación manual. Pendiente para iteración futura.
6. **Gestión de reservas en admin:** El panel de administración no incluye una vista para que el administrador cancele, modifique o gestione reservas de usuarios. El flujo actual es solo del lado del huésped.